

Challenge:	Riddle's Diary
Category:	Web Exploitation / SSTI / Cryptography
Difficulty:	Medium
Tools Used:	Browser, CyberChef or manual Vigenère decoder

Summary

This challenge demonstrates how web applications using server side templating can be exploited when user input is rendered without proper sanitization. By probing a magical diary interface we discover it is vulnerable to Server Side Template Injection (SSTI) using the Jinja2 engine. A keyword blacklist guards the system but it can be bypassed by splitting forbidden words into fragments and stitching them back using Jinja2's `|attr()` filter. This allows us to read a hidden file containing an encoded string. The string is decrypted using a Vigenère cipher with a key drawn from Harry Potter lore yielding a password that unlocks a ZIP archive containing the final flag.

Problem Statement

A recovered artifact - believed to be linked to Tom Riddle's Diary—has been digitized. It accepts written input and responds immediately. At first glance, it appears harmless.

But its responses suggest something deeper—something reactive.

Certain words are forbidden. Certain patterns are rejected. The system is guarded. But not perfectly.

Some protections yield not to brute force, but to a defensive charm—one whispered in the face of darkness.

What is written is rendered.

What is rendered... may reveal more than intended.

Steps to Solve

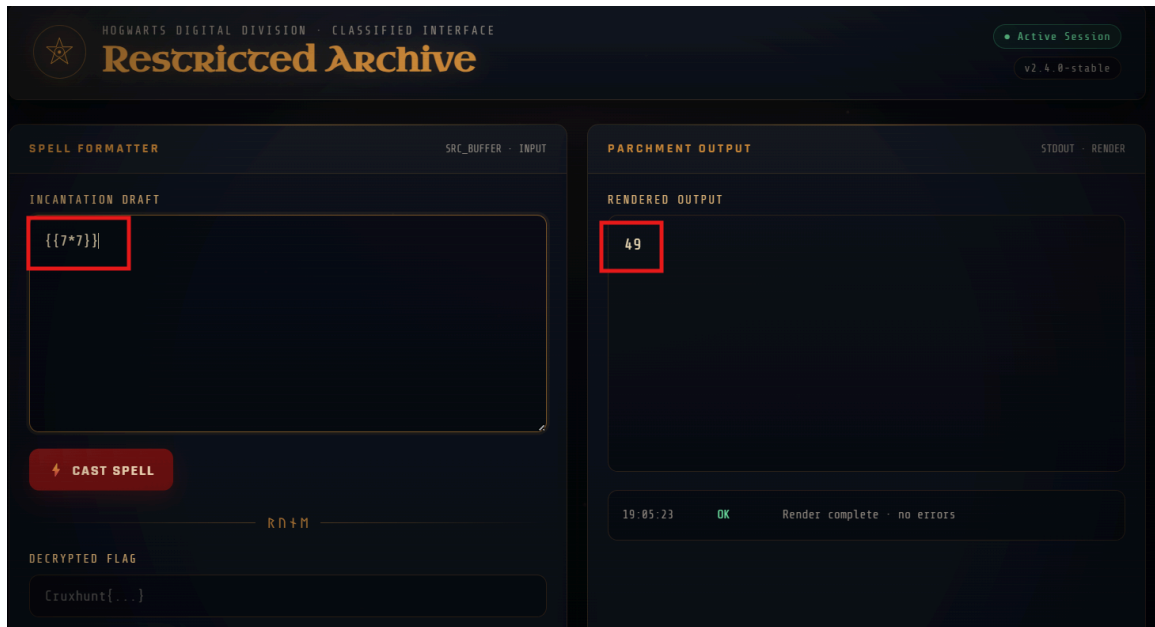
1. Before trying anything, re-read the problem statement:

"What is written is rendered" the server processes your input, it doesn't just display it

Hint 1 connection: "The diary doesn't just show your words, it actually runs them. Try giving it something that looks like a calculation."

These two clues together point to Server Side Template Injection (SSTI) — a vulnerability where user input gets passed into a template engine and executed as code.

2. Template engines use special delimiters to mark expressions that should be evaluated. The most common delimiter in Python based web apps is `{{ }}`.
So the standard first test is: `{{7*7}}`
3. The server returned 49, and it evaluated the expression. This confirms Server-Side Template Injection. The app is not treating your input as plain text, it is passing it through a template engine that executes it.



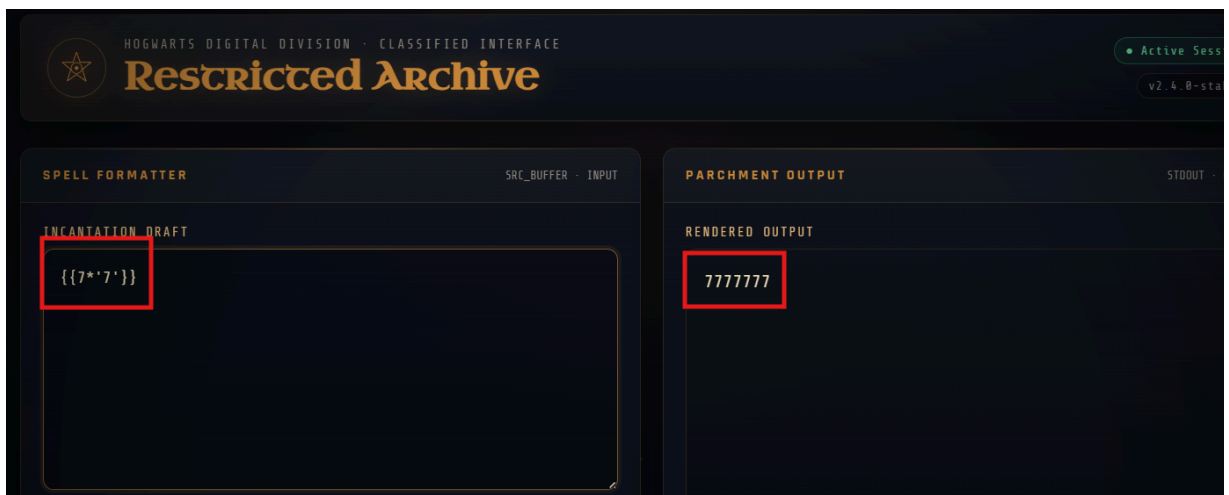
4. Knowing SSTI exists is not enough, we need to know which engine is running because each engine has different syntax and exploit chains.

Enter:

`{{7*7}}`

Output: 7777777

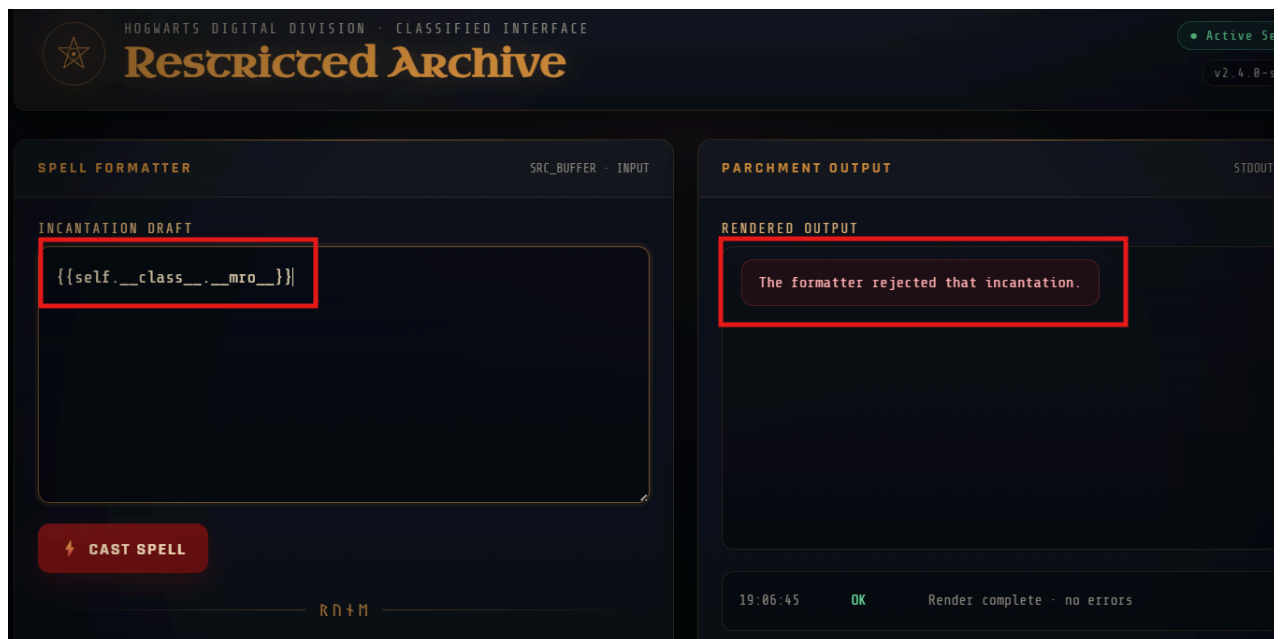
5. Getting 7777777 confirms that the engine is Jinja2, a Python based templating engine commonly used with Flask web apps.



6. Now that we know it's Jinja2, we try a standard payload to start navigating Python's object hierarchy:

`{{self.__class__.__mro__}}`

Output: The formatter rejected that incantation.

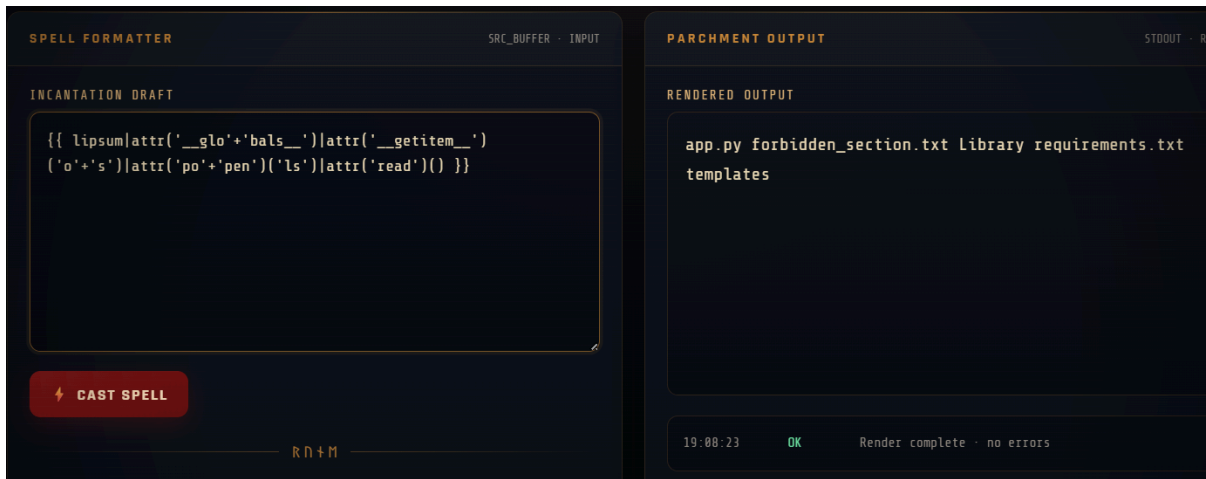


7. The server is actively blocking certain keywords. Words like self, class, mro, os, popen and globals are all blocked, the application checks your input for these substrings and rejects the request if found.

Hint 2 Connection: "The castle forbids certain words. But a forbidden word split in two... is no longer the same word, is it?" This confirms a blocklist exists and directly tells you the bypass strategy, split the words.

8. The blocklist works by scanning the raw input string for forbidden substrings before it reaches the template engine.
9. Jinja2 evaluates string concatenation at runtime, after the blocklist has already checked the input. So '___glo'+'bals__' passes the filter as two innocent fragments, but becomes ___globals__ when Jinja2 runs it.
10. We also replace dot notation (.attribute) with Jinja2's |attr() filter:
 lipsum.__globals__ → blocked (dot notation + keyword)
 lipsum|attr('___glo'+'bals__') → allowed
 lipsum is a built-in Jinja2 global function that generates Lorem Ipsum placeholder text. It's harmless-looking but because it's a global function, it carries a reference to Python's __globals__ dictionary which contains the os module. It's a perfect entry point that avoids needing self, config or request.
11. Hint Connection Hint 3: "There is a magical being that conjures dummy text. It knows all the secrets of this world." The "magical being that conjures dummy text" is lipsum. The "secrets" are the globals it carries.
12. Now use the bypass technique to run a shell command and list files:

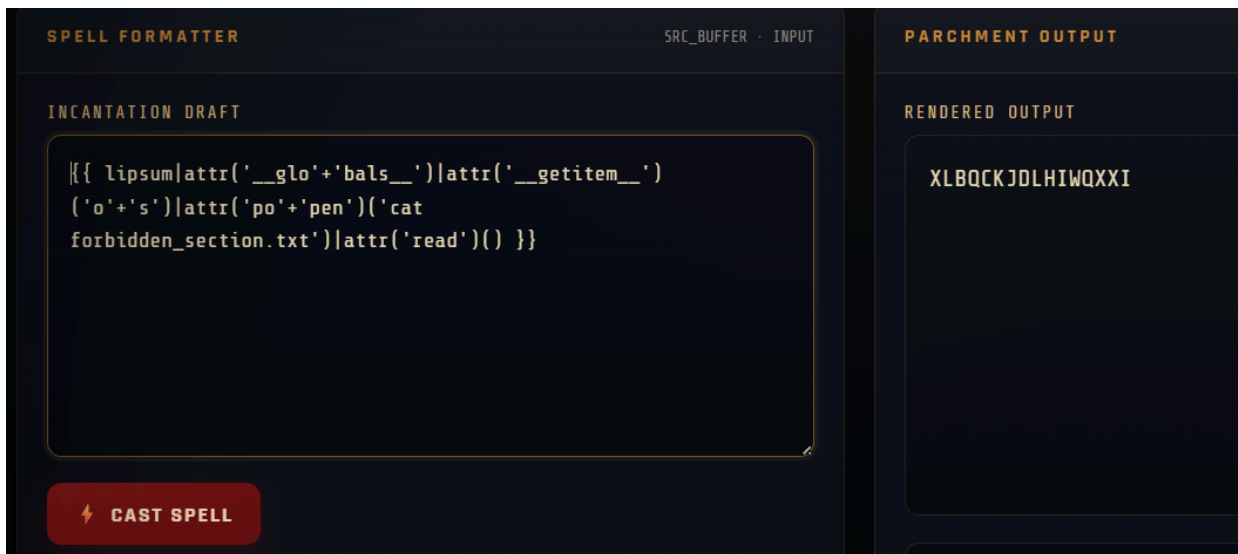
```
{{lipsum|attr('___glo'+'bals__')|attr('___getitem___')('o'+s')|attr('po'+pen')('ls')|attr('read')}()}}
```



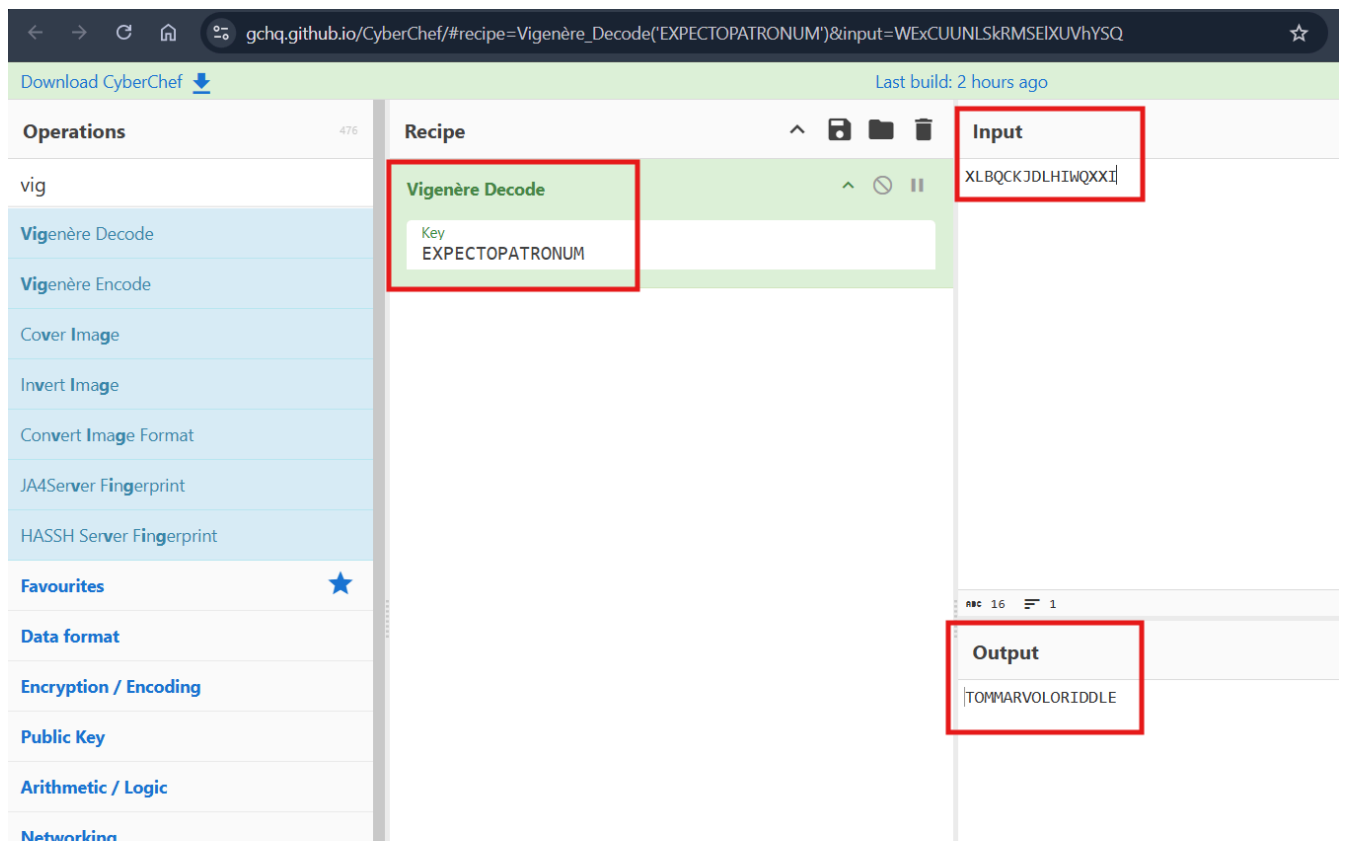
13. Among the listed files, forbidden_section.txt is visible and is clearly the target.
Now read the file using:

```
{{ lipsum|attr('__glo'+bals__)|attr('__getitem__')('o'+s')|attr('po'+pen')('cat
forbidden_section.txt')|attr('read')() }}
```

“Used attr() chaining to bypass blocklist and access os.popen”



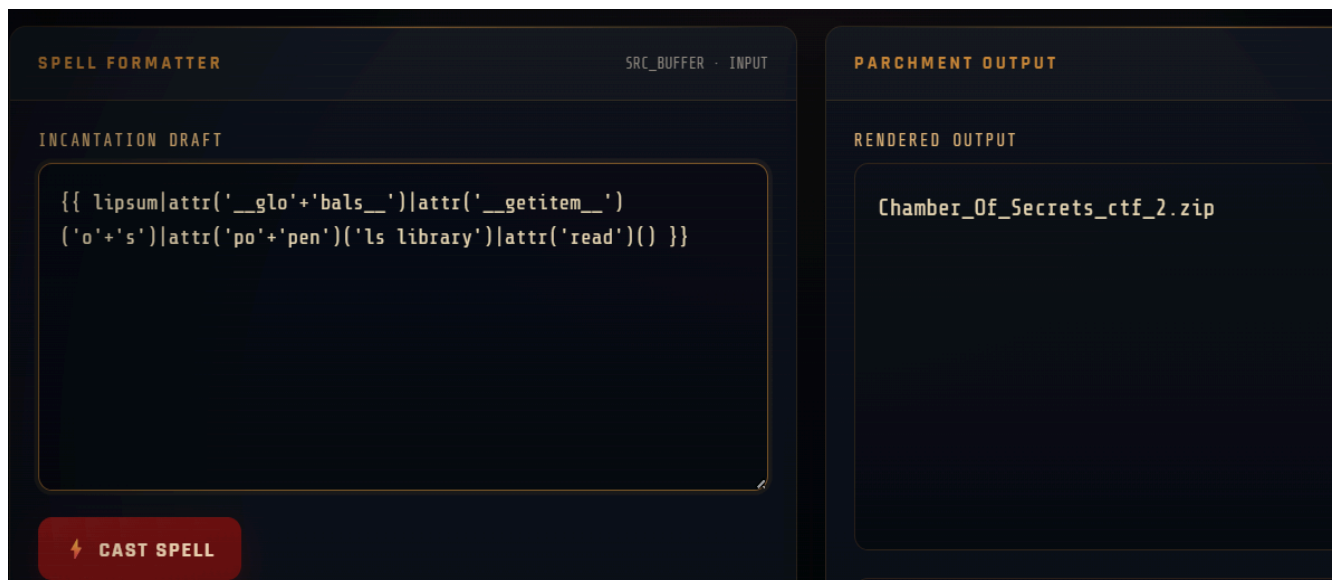
14. The output gives us an encoded string. The encoded string **XLBQCKJDLHIWQXXI** is encrypted using a Vigenère cipher — a classical substitution cipher where each letter is shifted by a corresponding letter in a repeating key.
15. The key is **EXPECTOPATRONUM** — the Harry Potter protective spell Expecto Patronum, referenced by the problem statement: "a defensive charm whispered in the face of darkness."
16. To decode, use [CyberChef](#) or [dcode.fr](#):
Input: XLBQCKJDLHIWQXXI
Key: EXPECTOPATRONUM
Operation: Vigenère Decode
Output: TOMMARVOLORIDDLE



Used CyberChef here

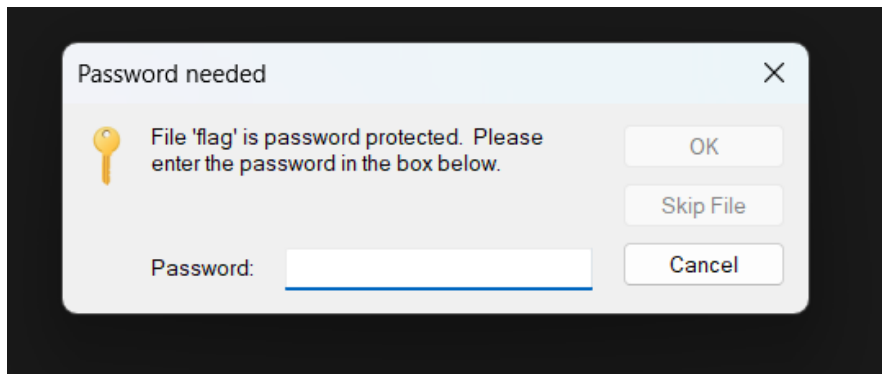
17. Now read other files too, you will discover that a ZIP file exists on the server from the ls output i.e

```
{{ lipsum|attr('__glo'+bals__)|attr('__getitem__')('o'+s')|attr('po'+pen')}('ls library')|attr('read')}() }}
```



18. Since web servers often expose static files directly, we can try accessing it via the browser:
https://<challenge-url>/Library/Chamber_Of_Secrets_ctf_2.zip
 The file gets downloaded

19. After that, open the ZIP file, a dialog box appears asking for the password, use the password decoded from the Vigenère cipher
 i.e TOMMARVOLORIDDLE



20. Finally, the flag appears after opening the flag.txt file

```
|cruxhunt{h0rcrux_d14ry_p4tr0nus_r3v34l3d}
```